

How the Pipeline Works

Interview → Route → Fetch / Build / Adapt → Validate

Talk to Galaxy · `dejian_workflow/` · NIAID-BRCs AI Codeathon 2.0

The one-line version

A free-text research description goes in. A real, mechanically-validated Galaxy workflow file comes out — reused, adapted, or built from scratch, never just a plausible-looking guess.



Driven end-to-end by `scripts/run_pipeline.py`.

Stage 1 — Interview

A plain text file, in the researcher's own words:

"I have Illumina paired-end amplicon sequencing of SARS-CoV-2 samples and want variant calls and lineage assignment with pangolin."

No structured form, no dropdown of organisms — this is deliberately the messiest input in the pipeline, because that's what a real interview produces. Three synthetic examples ship in `interviews/raw/`, plus 50 real ones — next.

Getting real interviews — what didn't work

Goal: unscripted real bioinformatics questions, not text written to make the pipeline look good.

Tried and failed, in order:

1. **Scrape biostars.org search results directly** (`curl`) — blocked.
Every request returns a Cloudflare bot-challenge page, not content.
2. **The official, documented Biostars API** — also blocked. Same Cloudflare challenge, even on the documented API path.
3. **Claude's own web-fetch tooling** — also blocked, HTTP 403.
4. **Defeat that bot protection anyway** — considered, **declined**.
Cloudflare's challenge is a deliberate access control the site put up; circumventing it isn't something this project does, no matter how few records were needed (50, here). Not a scope negotiation.

Getting real interviews — what actually worked

A published, openly-licensed dataset — not a scrape:

Luna, Augustin. (2023). *BioStars Posts API Output* [Data set]. Zenodo.

<https://doi.org/10.5281/zenodo.7813785> — CC BY 4.0, the same license

Biostars uses for its own content.

976 MB JSON, 532,421 entries (all post types) — a legitimate download of published research data, not a workaround.

Filtering stage	Count
Total entries (all post types)	532,421
<code>type == "Question"</code>	106,395
Matches a bacteria/virus/pathogen keyword	5,250
Scores as workflow-shaped (not troubleshooting/conceptual)	200
Hand-selected, diverse, genuinely workflow-shaped	50

Source-linked index: `interviews/BIOSTARS_SOURCES.md` .

What running the router against real data revealed

50 real interviews → **34 adapt / 13 build / 3 use**. Real language rarely crosses the confident "use" threshold — a useful, honest calibration signal three synthetic examples could never have given us.

One confirmed false positive, caught by the safety net:

"I am looking to make Hybrid assembly of a viral genome... paired-end reads from Illumina and long reads from MinION..."

→ routed **"use"** → `bacterial-qc-contamination-post-assembly` (0.38)

Wrong organism, at "use" (no-review-needed) confidence. But `generate_followup_questions.py` caught it anyway:

[organism] *This candidate assumes organism/target: bacteria. Your description didn't mention a matching term — is that correct...?*

Real validation that Goal 4's mechanism catches silent misroutes — not a constructed example. Also surfaced a genuine catalog gap: several real requests want generic bacterial variant calling, not yet in the catalog.

Classifying all 5,250 real questions

Before picking the 50, we classified the *entire* bacteria/virus/pathogen-matched set — not to filter further, but to see the real distribution of how people actually ask.

Category	Count	%
Analysis request (data + a goal)	2,869	54.6%
Other / unclear	1,714	32.6%
Tool recommendation (no own data)	272	5.2%
Troubleshooting / error	206	3.9%
Conceptual / definitional	107	2.0%
Data retrieval	54	1.0%
Installation / setup	28	0.5%

Better than expected: **over half are analysis-request-shaped**. But a third land in "other/unclear" — real questions blend categories more than any clean taxonomy admits.

Three shapes, three implications

1. Clean analysis request (54.6% — what the pipeline is built for)

*"I am working on 10 bacterial genomes (1 reference, 9 mutant), Illumina.
My aim is to find SNPs common in 9 genomes but absent in the reference..."*

2. Vague, needs clarification (tool recommendation, 5.2%)

*"I would like to classify my viral contigs, could anyone recommend the
best way? Also, has anyone made a viral database for blast?"*
— exactly what `generate_followup_questions.py` should catch, not guess.

3. Troubleshooting — out of pipeline scope entirely (3.9%)

"Error running AMR prediction... `Traceback...` `OSError: No such file or directory`" — ironically an AMR/TB request, but debugging a broken run, not describing a new analysis. No routing decision should apply here.

Stage 2 — Route (`route.py`)

```
route.py
└─ search_pipeline_catalog.py
    └─ knowledge_base/pathogen_genomics/galaxy_pipeline_catalog.yaml
        (20 curated bacteria/virus IWC workflows)
```

1. Tokenize the description
2. Score every catalog entry by **IDF-weighted keyword overlap** — a rare, specific term ("cgmlst," "pangolin") counts far more than a common one ("illumina," "paired")
3. Take the top match's **confidence** (0–1) and threshold it:

Confidence	Decision
≥ 0.35	use — strong, specific match
0.15 – 0.35	adapt — plausible but partial match
< 0.15 / no match	build — nothing close enough in the catalog

When this lands on **build**, one more check happens before generating anything from scratch — see next slide.

Stage 2 (extended) — checking the live registry

```
route.py: local decision == "build"
    ↳ galaxy_mcp_client.py (real MCP client)
        ↳ galaxyproject/galaxy-mcp (subprocess, stdio transport)
            ↳ the full, live IWC registry – real BM25 search
```

Real MCP protocol, not a reimplementation. Verified honestly — a genuine win **and** a genuine miss:

- **Win:** "differential expression analysis of mouse RNA-seq data" — 0 local catalog hits, but 3 real matches live (top: *RNA-Seq Differential Expression Analysis with Visualization*, BM25 score 18.37)
- **Miss:** a real viral-assembly request top-matches *Single-Cell Mixture Analysis: baredSC* at a **higher** score (98.47) — BM25 scores aren't confidence-calibrated across queries, unlike the local catalog's IDF ratio

So it only **surfaces** candidates for a human to review — never auto-upgrades the decision (`run_pipeline.py` 's `--force-build` overrides this deliberately, not silently).

Why IDF weighting, not raw keyword counts?

Raw overlap counts let generic terms dominate. A vague request sharing "illumina," "paired," and "end" with ten different workflows would outscore a precise one.

```
"...SARS-CoV-2 samples... lineage assignment with pangolin"  
→ matched: amplicon, assignment, cov, end, illumina,  
            lineage, paired, pangolin, sars, variant  
→ confidence 0.50 (dominated by rare terms: pangolin, lineage, sars)
```

The catalog entry's own confidence is *matched-weight ÷ that entry's total keyword weight* — comparable across entries with different-length keyword lists, not just a raw count.

Where this sits in the landscape

What we use — both real, standard IR techniques, not invented for this project:

- **Local catalog:** IDF-weighted keyword overlap — the "IDF" half of TF-IDF, a classic technique since Spärck Jones (1972)
- **Live IWC registry:** BM25 (via `galaxy-mcp`) — the industry-standard ranking function behind Elasticsearch, Solr, and Lucene

Neither is LLM-based — both are exact-token keyword techniques, chosen for this v0 because they're auditable, deterministic, and free to run: every match comes with an explicit "matched on: cgmlst, pangolin" trail, not a black-box score.

The cost of that choice: exactly the paraphrase-blindness the biostars-134625 miss demonstrated — a description can't match a workflow whose keywords it never literally uses.

Other ways to match text ↔ workflow

Alternative	Gains	Cost
TF-IDF cosine similarity	More textbook than our ratio	Same synonym-blindness
Fuzzy string matching	Catches typos/misspellings	Not synonyms or paraphrase
Dense embeddings	Fixes "flu" vs. "influenza" — semantic, not literal	Needs an embedding model/API call
Hybrid sparse + dense	Best of both — current RAG standard	More moving parts
LLM-based classification	Handles paraphrase/reasoning	Non-deterministic, costs a call, harder to audit
Structured field extraction	Precise organism/data-type matching	Needs a taxonomy + reliable extractor

Embeddings are the natural next step if paraphrase-blindness turns out to matter more than auditability — nothing here rules that out later.

Stage 2b — Follow-up questions (Goal 4)

```
route.py → generate_followup_questions.py
```

For each routed candidate: did the description actually confirm organism / data_type / analysis_type, or is it a silent assumption?

Anything with **zero matched keywords** becomes a question, not a guess.

Real example (E. coli cgMLST request, routed to "adapt", confidence 0.21):

1. assumes **bacteria** — description didn't confirm
2. assumes **assembly/FASTA** — description didn't confirm
3. assumes **strain typing** — description didn't confirm
4. only a partial match — what's actually different?

Deterministic, not LLM-based — needs no credentials, only surfaces gaps that provably exist in the matching data, never fabricated concerns.

Known limit: surfaces questions, doesn't collect answers or re-route on them — the interview loop that closes this isn't built yet.

Stage 3a — Use path

```
fetch_iwc_workflow.py → validate_workflow.sh
```

- Lists the workflow's real directory on GitHub via the API (IWC has **no consistent filename convention** — guessing from the directory name silently fails on ~1/3 of workflows)
- Downloads the actual `.ga` + Planemo test file
- Runs `planemo workflow_lint` — checks tool ids, version/changeset installability, and step connections against the **live Galaxy Tool Shed**, no Galaxy server required

This already caught a real bug: a published SARS-CoV-2 workflow in IWC currently references an uninstallable toolshed changeset revision.

Stage 3b — Adapt path

Fetches the same real base workflow, then stops — deciding *what* to change from free text needs a human or an LLM, and this script doesn't fabricate that decision. Instead it takes a **change spec**:

```
[{"step_id": "2",  
  "tool_state_path": ["scannew_section", "min_id_new_allele"],  
  "new_value": "95",  
  "note": "researcher wants a stricter identity threshold"}]
```

```
adapt_workflow.py propose workflow.ga change_spec.json # review the diff  
adapt_workflow.py apply workflow.ga change_spec.json out.ga
```

`apply` deep-copies the workflow, edits only the named steps, then **byte-compares every other step against the original** — refuses to write output if anything outside the change spec differs.

Stage 3c — Build path

No workflow in the catalog is close enough → generate one.

```
build_workflow.py
└─ anthropic SDK (claude-opus-5)
   └─ validate_workflow.sh (same real check as the use path)
```

- The LLM is told to only use tool ids it's genuinely confident exist
- Generation is **not trusted on its own** — the output goes through the exact same `planemo workflow_lint` check as a fetched workflow
- A generated workflow that fails validation is reported as a **failure**, not silently written as "done"

This is the one path most exposed to confident fabrication — which is exactly why it's graded by the same program as everything else, not exempt from it.

Stage 4 — Test execution (separate step)

```
run_workflow_tests.sh workflow.ga
```

Runs `planemo test` — executes the workflow against real data, not just structural checks.

Mode	Where it actually executes
<code>GALAXY_URL</code> + <code>GALAXY_USER_KEY</code> (preferred)	Remote — e.g. usegalaxy.org's own compute
<code>GALAXY_ROOT</code>	Local — a Galaxy checkout on your machine
neither set → <code>--install_galaxy</code>	Local — a disposable Galaxy Planemo downloads

Static validation (stage 3) only checks the workflow is *well-formed*. This is the step that checks it actually *does the right analysis*.

Where does the data actually go?

This is not just a speed tradeoff — it's a data-location decision.

- **Online (`GALAXY_URL / GALAXY_USER_KEY`)**: your machine only orchestrates. The workflow and test data are **uploaded to that Galaxy server**, tools run on **its compute**, and results are downloaded back. Fast, zero local setup — but your data leaves your machine and is processed by a third-party public service.
- **Local (`GALAXY_ROOT` or `--install_galaxy`)**: Galaxy and every tool run **entirely on your machine**. Nothing is uploaded anywhere. Slower to set up, but your data never leaves.

Rule of thumb: fine for the example/test data this pipeline ships with.

For real, sensitive, or restricted research data, use the local mode.

What's real vs. what's a stand-in

Piece	Status
Catalog, routing, fetch, static validation, byte-stable adapt	Real , tested against live GitHub + Galaxy Tool Shed
Live IWC registry check (build path)	Real MCP integration (galaxy-mcp) — surfaces candidates, doesn't auto-decide
Follow-up questions	Real, deterministic — surfaces gaps, but no answer-collection loop yet
Build path	Real LLM call + real validation, but no live Galaxy Workflow Foundry integration
Adapt path's "what to change"	Needs a human/LLM-authored change spec — no NL interpretation yet
Catalog coverage	~20 curated locally, plus the full live IWC registry as a fallback check

None of this is faked to look more finished than it is — see each script's own docstring and `SETUP.md` for the honest gaps.

Try it

```
cd dejian_workflow
pip install -r requirements.txt
python3 scripts/run_pipeline.py interviews/raw/example-sarscov2-amplicon.txt
python3 scripts/run_pipeline.py interviews/raw/example-ecoli-outbreak-cgmlst.txt
```

Full install/credentials/cleanup instructions: `SETUP.md`

Script-by-script reference: `scripts/README.md`

Full project proposal: `PROPOSAL.md`